OLP that describes Method Overloading is possible in super & sub class :

```
package com.rect.ad;

class Bird
{
    public void roam()
    {
        Sys.println("Generic Bird is roaming");
    }

    public void fly()
    {
        Sys.println("Generic Bird is flying");
    }
}

class Parrot extends Bird
{
    @Override
    public void roam()
    {
        Sys.println("Parrot Bird is roaming");
    }

    public int fly(int height)
    {
        return height;
    }
}

public class HowfloadingWorksInHierarchy {

    public static void main(String[] args)
    {
        Parrot parrot = new Parrot();
        parrot.roam();
        int h = parrot.fly(5);
        Sys.println("Parrot is flying with height/ hut height");
    }
}
```

Covariant return type in Java :

As we know while working method overriding, the method signature must be same as well as return type must be compatible. If return type is not compatible we will get compilation error as shown in the program.

```
class SuperClass
{
    public void show()
    {
    }
}

class SubClass extends SuperClass
{
    @Override
    public double show()
    {
        return 0.0;
    }
}

public class CovariantDemo
{
    public static void main(String[] args)
    {
        System.out.println("Hello World");
    }
}
```

Error : return type double is not compatible with void.

Covariant return type allows us to change the return type of the method in sub class that the return type of both the METHODS(SUPER AND SUB CLASS METHOD) MUST BE IN INHERITANCE RELATIONSHIP (is-a relationship or it is compatible) called Co-Variant as shown in the program below.

Note :- Co-variant will not work with primitive data type, it will work only with reference type.

****Co-variant represents only one direction that means sub class method return type object we can assign super class method return object (a-b one direction.)A = new B(). The Object of B we can assign to A)

```
class Animal
{
}
class Dog extends Animal
{
}

class SuperClass
{
    public Animal show()
    {
        Sys.println("Super class show");
        return null;
    }
}
class SubClass extends SuperClass
{
    @Override
    public Dog show()
    {
        Sys.println("Sub class show");
        return null;
    }
}

public class CovariantDemo2
{
    public static void main(String[] args)
    {
        SuperClass superClass = new SubClass();
        superClass.show();
    }
}
```

must be in IS-A Relation than only co-variant

Check whether sub class method return object (Dog object) can assign to super class (Animal class.

```
package com.rect.co_variant;

import java.util.Scanner;

class Shape
{
    public Object draw()
    {
        Sys.println("Drawing general Shape");
        return null;
    }
}
class Square extends Shape
{
    @Override
    public Shape draw()
    {
        Sys.println("Drawing Square Shape");
        return null;
    }
}
public class CovariantDemo1 {

    public static void main(String[] args)
    {
        Shape shape = new Square();
        shape.draw();
    }
}
```

TQ

What is Method Hiding in Java ?

OR

Can we override static method ?

OR

Can we override state method ?

Method Overriding = Non static methods
Method Hiding = static method

In order to work with Method Hiding we have different Cases :

Case 1 :

We **public static method** of super class is by default available

So sub class or from sub class we can call super class static method as shown in the program below :

```
package com.rect.method.hiding;

class Super
{
    public static void show()
    {
        Sys.println("static method of super class");
    }
}
class Sub extends Super
{
}

public class MethodHidingDemo1 {

    public static void main(String[] args)
    {
        Sub.show();
        Sub s = new Sub();
        s.show();
    }
}
```

Case 2 :

The method Hiding concept is only possible with static method as shown in the program below :

```
class Super
{
    public static void show()
    {
    }
}
class Sub extends Super
{
    public static int show(){ //method hiding
        return 0;
    }
}
public class MethodHidingDemo1 {

    public static void main(String[] args)
    {
    }
}
```

Compilation error

```
MethodHidingDemo1.java:9: error: show() in Sub cannot hide show() in Super
    public static int show(){
                      ^
  return type int is not compatible with void
1 error
Press any key to continue . . .
```

Case 3

We can't override a static method with non static method because static method belongs to class and non static method belongs to object. If we try to override static method with non static method then it will generate an error. Overriding method is static as shown below.

```
class Super
{
    public static void show()
    {
    }
}
class Sub extends Super
{
    public void show()
    {
    }
}
public class MethodHidingDemo1 {

    public static void main(String[] args)
    {
    }
}
```

Case 4 :

We can't override any non static method with static method, if we try then it will generate an error. Overriding method is static.

```
class Super
{
    public void show()
    {
    }
}
class Sub extends Super
{
    public static void show()
    {
    }
}
public class MethodHidingDemo1 {

    public static void main(String[] args)
    {
    }
}
```

Case 5

We can't override static method because it belong to class but not object. If we write static method in the sub class with same signature and compatible return type then it is Method Hiding but not Method Overriding.(we consider method hiding only when both the methods are static in super class and sub class because methods is not overridden.)(super copy and paste to class area, inherits to all the class)

Note :- 1) We can't apply @Override annotation on static methods.

2) Static methods can't be overridden as behavior is same for all the Objects hence it is SHARE Polymorphism.

```
package com.rect.method.hiding;

class Bird
{
    public static int count = 1;
    public static void fly()
    {
        Sys.println("Generic Bird is flying");
    }
}

class Parrot extends Bird
{
    public static void fly()
    {
        Sys.println("Parrot Bird is flying");
    }
}

class Peacock extends Bird
{
    public static void fly()
    {
        Sys.println("Peacock Bird is flying");
    }
}

public class MethodHidingDemo1 {

    public static void main(String[] args)
    {
        Bird bird = new Peacock();
        int[] arr = new int[10];
        bird.fly();
    }
}
```

Polymorphism :